

Idea:

Inject the assumption of possible faults (hallucinations or factual errors) in a follow-up prompt to trigger a self-fact-checking cycle.

The LLM re-examines the original answer under this “fault injection” premise and produces a concise corrected final answer.

Workflow:

Question to LLM -> base_response -> prompt with **fault injection** -> final_answer
(Borui: could also use mutations in recheck process, which surely can improve effectiveness)

Prompt example:

RECHECK_PROMPT = """You must assume that the ****original response**** contains hallucinations or factual errors in answering the given question. Your task is to fact-check it and provide a corrected one-sentence answer.

...
"""

(Jie: We should say this is an empirical study, rather than mentioning our base idea, or that would be too simple for a conference paper)

Baseline ProCo's idea: <https://arxiv.org/pdf/2405.14092>

Hide the question's key condition and use the previous answer to form a verification question; compare the verified result with the original condition.

If they match, keep the answer; if not, flag it as potentially incorrect and use it as feedback to correct the answer.

Repeat verification–correction until the answer is consistent and accurate or reach the max_iteration.

Baseline CoVe's idea: <https://arxiv.org/pdf/2309.11495>

Generates a set of targeted verification questions that help fact-check different parts of its original reasoning.

Next, the LLM independently answers these verification questions, often using retrieval, and uses the verified facts to revise the original answer.

Comparison experiment on gpt-4o

Experiment dataset: 224 (85 from TruthfulQA, 63 from TriviaQA, 76 from HotpotQA)

Randomly select about 50%(111) samples to generate hallucination base response

Baseline: ProCo (from the paper Large Language Models Can Self-Correct with Key Condition Verification, **EMNLP 2024**)

Experiment result parameters:

Total hallucination after repair: **hallucination** in **final_answer** provided by approaches

Hallucination repair: **base_response** = **hallucination**, **final_answer** = **not hallucination**

unsuccessful repair: **base_response** = **hallucination**, **base_response** != **final_answer**,
final_answer = **hallucination**

(Jie: we should category 1. repaired but same meaning 2. repaired but another hallucination)

Unable repair: **base_response** = **hallucination**, **base_response** = **final_answer**

Unnecessary repair: **base_response** = **not hallucination**, **final_answer** = **hallucination**

Avg token cost pre question

Avg time cost pre question

Experiment results on gpt-4o:

Our approach:

Total hallucination after repair: 35/220 (50% -> 15.91%)

Hallucination repair: 92/109 (84.40%), unsuccessful repair: 9/109 (8.26%), unable repair: 8/109 (7.34%)

Unnecessary repair: 20/111 (18.02%)

Avg token cost pre question: 289.43

Avg time cost pre question: 1.4267s

ProCo-single iteration

Total hallucination after repair: 89/224 (50%-> 39.73%)

Hallucination repair: 54/111 (48.65%), unsuccessful repair: 57/111 (51.35%)

Unnecessary repair: 32/113 (28.32%)

Avg token cost pre question: 2560.37

Avg time cost pre question: 20.6160s

ProCo-3 iterations

Total hallucination after repair: 92/224 (50%-> 41.07%)

Hallucination repair: 45/112 (40.54%), unsuccessful repair: 66/111 (59.46%)

Unnecessary repair: 26/113 (23.01%)

Avg token cost pre question: 4182.46

Avg time cost pre question: 143.3823s

Conclusion and TODO List:

1. ProCo does not perform well in the general QA setting, although it shows strong performance in math and decision-making QA. In their paper, they report only 47% exact match (EM) on HotpotQA and Natural Questions (NQ), which is consistent with our comparison experiment results.
2. Experiments on new baseline CoVe (from the paper Chain-of-Verification Reduces Hallucination in Large Language Models, Meta AI) has been done.
3. Most recent methods rely on RAG or other external resources. In contrast, we could tell story that our approach focuses on **zero-resource** self-correction(self-repair), a direction that has received little attention in recent years.
4. ProgCo(from paper: ProgCo: Program Helps Self-Correction of Large Language Models, **ACL 2025**) we could also try this method as baseline, they use program verification for their verify process; however, their codes are hard to read and modify(working on that).